

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)
Department of Computer Science and Engineering

ASSESSMENT TOOL 2- Concept Mapping

Course Code:	CSA0306	Course Title:	Data Structures
CO Assessed:	CO1 – Imitation (BL3, BL4)	Assessment:	Assessment Tool 2 – Concept Mapping
Weightage:	10% of CIA	Total Marks:	100
CO1 Statement:	Basics, Data, Data object, Data structure, Abstract Data Types (ADT), realization of ADT in 'C', Primitive and non-primitive data structure, Linear and non-linear data structure Arrays & Recursion, Performance analysis and Measurement: Time and space analysis of algorithms, Average, best and worst-case analysis.		
Student Name:	Tejasvi M	Reg. No.:	192511041
Section / Batch:	2025-29	Date:	16/07/2026

Code of Conduct

I (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:



Instructions

- This test contains 2 concept mapping. Total: 100 Marks.
- Evaluation of Answer Quality -Checking whether the answer includes: Concept Identification , Linkages, Organization, Integration of Literature, and Clarity
- No DTP printouts or electronic devices permitted. They should provide materials in PDF format.

Rubrics

Questions	Problem Understanding	Method Selection	Solution Process	Accuracy of Calculation	Interpretation of Results	Total Marks
Q1						
Q2						

STUDENTS ATTENDANCE SYSTEM

A student attendance system records daily presence data. Construct a concept map linking array, fixed size, sequential access, linked list, dynamic allocation, flexibility.

SYSTEM OVERVIEW

The system stores and manages daily attendance of students. Data may grow or shrink as students join or leave.

ARRAY

HOW IT WORKS: Stores elements in continuous memory. Each element is accessed using index.

DIAGRAM

Index: 0 1 2 3 4
Student: A B C D E

PROS:

- Random access
- Simple implementation

CONS:

- Size is fixed
- Insertion / Deletion costly

USE: Good when no. of studs is fixed and not changing.

FIXED SIZE

HOW IT WORKS: The size of the structure is decided at the time of creation.

EXAMPLE: (ARRAY)

Size 5, only stores 5

1 1 1 1 1

PROS:

- Predictable memory usage
- Easy to manage

CONS:

- Cannot handle more data
- Less adaptable

USE: Suitable when no. of max studs known.

SEQUENTIAL ACCESS

HOW IT WORKS: Elements are accessed one by one in order.

DIAGRAM:

one by one

A → B → C → D → E

PROS:

- Easy traversal
- Simple searching

CONS:

- Slows for large data

USE: Daily attendance is usually processed in order, so sequential access.

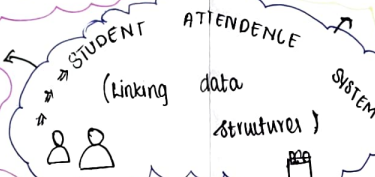
OBJECTIVES

- Stores daily attendance records efficiently

- Allow easy insertion, deletion and updates

- Handle changing number of students.

- Choose the best data structure for data updation



Key decision

For dynamic data updates, linked list is more suitable because of its dynamic allocation and flexibility.

Comparison table

Flexibility - Size
Linked list - Insertion

LINKED LIST

HOW IT WORKS: Stores element in nodes. Each node contains data.

Diagram:

Head → A → B → Null

PROS:

- Dynamic size
- Easy insertion

CONS:

- No random access
- Extra memory

USE: Ideal when students can join or leave

Dynamic Allocation

HOW IT WORKS: Memory is attached at runtime as per requirement.

Example:

A → B → C → D

PROS:

- Memory used as needed
- Efficient for variable

CONS:

- Overhead of allocation
- Complex management

MOST SUITABLE FOR DYNAMIC DATA UPDATES

Linked List

- Support Dynamic Allocation
- Easy Insertion / Deletion
- Flexible size
- No continuous memory required
- Efficient for changing data

JUSTIFICATION

- In a student attendance system, number of students can change daily.
- Array has fixed size → may waste space.

FLEXIBILITY

HOW IT WORKS: The structure can adapt to changes such as insertions.

IN LINKED LIST:

Add / Remove nodes anywhere without affecting others.

PROS:

- Highly adaptable
- Support frequent

CONS:

- More complex the array.

Q: A payroll system processes employee salaries sequentially. Construct a concept map connecting Array, sequential Access, Processing Efficiency, limitations. Explain how sequential processing works and justify its advantages.

ARRAY

An array is a collection of similar elements stored in contiguous memory locations

KEY POINTS :

- store employee data of fixed size
- each record can be accessed using an index
- elements are stored in a continuous block of memory
- Ideal for processing data in a fixed order



PAYROLL SYSTEM OVERVIEW

- A payroll system calculates and distributes salaries to employees.
- employee records are stored and processed

CONCEPT MAP

PAYROLL SYSTEM
PROCESSES EMPLOYEE
SALARIES SEQUENTIALLY



PROCESSING

EFFICIENCY

- sequential processing using arrays is efficient for Payroll system

TIME COMPLEXITY

visiting all n employees once

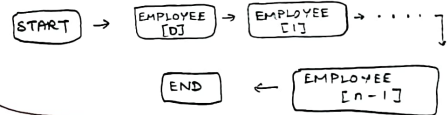
$$\rightarrow O(n)$$

where n is number of employees

HOW SEQUENTIAL PROCESSING WORKS

1. Employee records are stored in an array
2. Start from the first employee (index 0)
3. Read employee details
4. Calculate salary (Basic + allowance - deductions)
5. Update total payroll amount
6. Move to next employee (index +1)
7. Repeat until last employee
8. Generate payroll summary & reports

FLOW OF SEQUENTIAL PROCESSING



LIMITATIONS :-

Not suitable for frequent insertions or deletion

Inefficient if only a few records need processing

FINAL SUMMARY

In a payroll system, employee data stored in an ARRAY is processed using SEQUENTIAL ACCESS. Each record is handled one after another, which ensures PROCESSING EFFICIENCY through simple and fast traversal. However, LIMITATIONS such as fixed size and inflexibility exist -